GraphRAG Explorer

Wie österreichische Banken, Eigentümer und Aufsichtsbehörden zusammenhängen

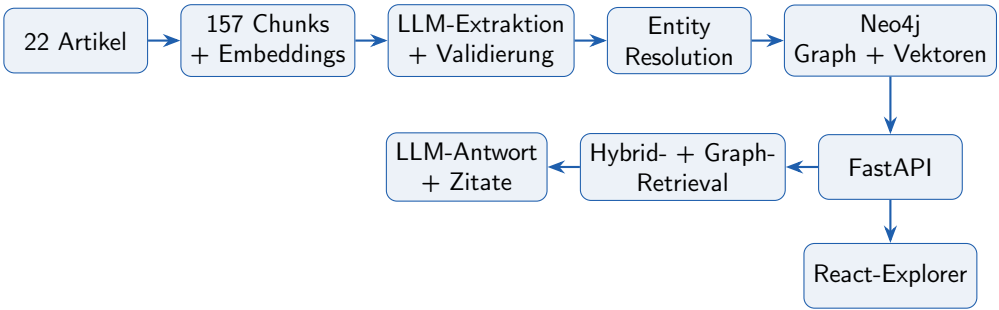
Kacper Zaleski

Das Ziel

- Eine **lauffähige App**, in der man Fakten und Zusammenhänge **visuell entdeckt**, die man nicht kannte
- Von Dokumenten über einen Knowledge Graph bis zur Explorer-UI, durchgängig
- Jeder Fakt ist **bis zur Quellpassage nachvollziehbar**
- Und die ehrliche Frage: Macht der Graph die **Antworten** besser als einfaches RAG?

Korpus	22 Wikipedia-Artikel: österreichische Banken, Eigentümer, Aufsicht
Stack	Python, FastAPI, Neo4j, React + Cytoscape.js, Docker
LLMs	Gemini (Free Tier), umschaltbar: Groq, Cerebras, Mistral

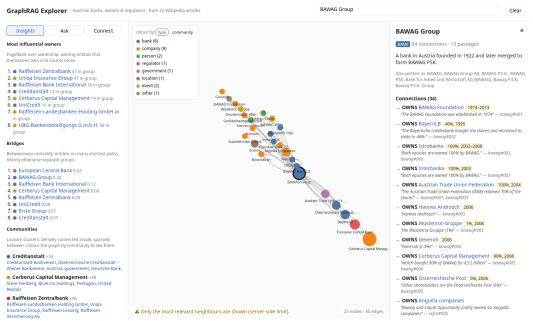
Architektur



- Die Ingestion ist **idempotent und gecacht**: ein erneuter Lauf macht 0 API-Calls
- Ergebnis: **726 Entitäten, 585 Relationen**, jede Kante kennt ihre Quell-Chunks

Live-Demo

1. Insights: einflussreiche Eigentümer, Communities
2. Entitätskarte: Eigentümergeschichte der BAWAG
3. Connect: ÖGB ↔ GoldenTree
4. Connect: Refco ↔ Cerberus
5. Ask: *Who have been the owners of BAWAG over time?*
6. Ask: *How is BayernLB connected to Addiko Bank?*



Extraktion: das LLM schlägt vor, der Code prüft

- Ein LLM-Call pro Chunk, **Pydantic-Schema** als Structured Output
- Feste Typlisten: 8 Entitätstypen, 9 Relationstypen
- Eine **kanonische Richtung**: „A ist Tochter von B“ wird zu `B OWNS A`
- Jedes Element wird einzeln validiert: eine fehlerhafte Relation verwirft nicht den ganzen Chunk
- **Grounding-Check**: das Belegzitat muss vom Chunk gestützt sein

Was schiefging: Exaktes Substring-Grounding verwarf 73 Relationen, davon ~70 korrekte (Zitate mit „. . . “). Mit Wort-Abdeckung bleibt 1 übrig, und es erkennt trotzdem, wenn das Modell Umlaute verschluckt („Österreichische“ → „sterr ischische“).

Entity Resolution: falsche Merges sind schlimmer als Splits

- Normalisieren: Groß-/Kleinschreibung, Akzente, ö→oe, Rechtsformen (AG, GmbH, a.s.)
- Danach eine **kuratierte Alias-Datei**: jeder Merge ist explizit und prüfbar
- Typ per Mehrheitsentscheid (das LLM sagt einmal BANK, einmal COMPANY)
- In einer Bank würde ein Merge zweier „Raiffeisen . . . “ **Eigentum erfinden**

Was schiefging: Die Kette ÖGB → BAWAG ← Cerberus war unterbrochen: das LLM schrieb in einem Chunk „Cerberus“ und „ÖGB“, in anderen die vollen Namen. Außerdem ist „Ceberus“ ein Tippfehler in Wikipedia selbst.

Retrieval: Graph-Modus

1. **Hybride Suche**: Vektor + Volltext, fusioniert mit Reciprocal Rank Fusion (Ränge statt Scores)
2. **Seeds**: Entitäten, deren Namen in der Frage vorkommen
3. **Pfade** zwischen den Seeds, dann Breitensuche mit Knotenlimit
4. Orte sind nie Zwischenschritte („beide sitzen in Wien“)
5. Chunks hinter den Kanten + Reranker; **jede Pfadkante hat einen reservierten Platz** für ihren Beleg
6. Antwort mit Zitaten; der Code entfernt Zitate von Passagen, die das Modell nicht gesehen hat

Was schiefging: Ein Hub-Filter nach Knotengrad blockierte die wichtigsten Banken: hier *sind* die Knoten mit hohem Grad die Antwort. Der Typ (Ort) war das richtige Signal.

Funktionen zum Entdecken

- **Zeit und Anteil an den Kanten**: ein zweiter, gezielter LLM-Durchlauf (15 Calls, rund 200 Kanten)
  BayernLB 46 % (1995) → ÖGB 100 % (2004) → Cerberus 90 % (2006) → GoldenTree 21,8 %
- **Graph-Analytik** bei der Ingestion (networkx):
  - PageRank über umgekehrte Eigentumskanten: wer kontrolliert am meisten
  - Betweenness: Brücken zwischen Gruppen (EZB, BAWAG)
  - Louvain-Communities: Erste + CEE-Banken, UniCredit + Bank Austria, …
- Pfadsuche ohne LLM: sofort und kostenlos

Was schiefging: „SIA“ war unter den Top-5-Eigentümern. Aus “UniCredit verkaufte ihr *Kartengeschäft* an SIA” war `SIA ACQUIRED UniCredit` geworden. Eine falsche Kante verzerrte eine globale Kennzahl.

Engineering für unzuverlässige Free-Tier-APIs

- Timeouts, Retries mit Backoff, clientseitiger **Rate Limiter**
- **Fallback-Kette** von Modellen bei 503 und bei Quota-Fehlern
- **Inhaltsadressierter Cache**: erneute Läufe sind kostenlos und reproduzierbar
- Alles-oder-nichts-Schreiben: ein fehlgeschlagener Lauf schreibt nichts und setzt aus dem Cache fort
- API: Ressourcen über Lifespan, Dependency Injection, Request-IDs, Liveness vs. Readiness

Was schiefging: `gemini-3.5-flash` erlaubt **20 Requests pro Tag**, die Ingestion braucht 160. Außerdem zählt der Free Tier jeden Text in einem Embedding-Batch als eigenen Request.

Evaluationsdesign

- 28 Fragen: 13 Multi-Hop, 10 Lookup, 5 korpusweite
- Erstellt aus dem **Text, nicht aus dem Graphen**: sonst würde der Graph-Modus an seinen eigenen Daten gemessen
- Metriken: Recall@8, Complete@8, LLM-Judge, Entity Recall, Latenz, Tokens
- Judge von einem **anderen Anbieter** (Groq) als die Antworten (Gemini): Self-Preference-Bias
- **Gepaarte** Statistik: Bootstrap-Konfidenzintervalle, McNemar, Wilcoxon
- Reliabilität des Judges: zweiter Judge (Qwen), Cohens κ

Runde 1 (ursprünglicher Prompt): kein messbarer Unterschied

	Vektor	Graph
Judge-Score	0,91 [0,80; 0,98]	0,91 [0,84; 0,98]
korrekt	24/28	23/28
Recall@8	0,90	0,89
LLM-Latenz p50 / Tokens	1,7 s / 2,8k	2,5 s / 3,4k

- Differenz +0,00 [−0,05; +0,05], McNemar p = 1,0
- **Deckeneffekt**: generierte Fragen nennen ihre Entitäten, die Vektorsuche findet beide Passagen
- Beide Modi scheitern an **korpusweiten** Fragen: Top-k kann keine Aggregation einsammeln

Runde 2: ein vorab registrierter, schwerer Test

- Hypothese **vor** dem Lauf festgelegt: der Graph gewinnt bei Verbindungsfragen
- 8 Fragen „How is A connected to C?“, die Brücke wird nirgends genannt
- Prompt-Fix nur auf diesen neuen Fragen evaluiert: **kein Tuning auf dem Testset**

	Vektor	Graph
Judge-Score	0,38	0,44
korrekt	3/8	2/8
Recall@8	0,56	0,50
Recall@8 **ohne Reranker**	**0,81**	0,56

- Differenz +0,06 [−0,12; +0,25]: **Hypothese nicht bestätigt**
- Der Reranker schadet Verbindungsfragen am meisten (Vektor-Recall 0,81 → 0,56)
- Regressionscheck auf den alten 28: nichts kaputt (Graph 0,93 vs. Vektor 0,91)
- Zwei Judges stimmen bei 51/56 + 13/16 Antworten überein, Cohens $\kappa = 0{,}62$ / 0,69

Was ich gelernt habe

Wo der Graph hilft

- Exploration: Pfade, Nachbarschaften, Rankings
- Erklärbarkeit: jede Kante hat ein Quellzitat
- Struktur: Zeit und Anteil bei Eigentum
- Fehler werden sichtbar (SIA, Cerberus)

Wo (noch) nicht

- QA-Genauigkeit auf einem kleinen, sauberen Korpus
- Fehlende Extraktionen begrenzen ihn: „Addiko wurde aus der Hypo abgespalten“ wurde nie zur Kante
- Globale Fragen brauchen Graph-Abfragen, nicht Top-k

Ein ehrlich gemessenes negatives Ergebnis ist auch ein Ergebnis.

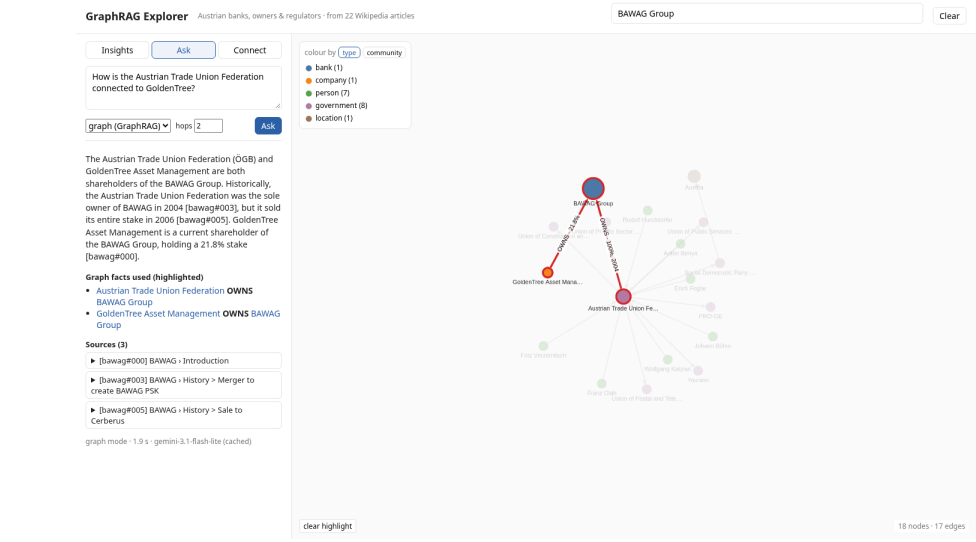
Nächste Schritte

- Reranker im Graph-Modus weglassen oder ersetzen (er ist auf Web-Suchanfragen trainiert)
- Aggregationsmodus für korpusweite Fragen (Graph-Abfrage statt Top-k)
- Höherer Recall bei der Extraktion: zweiter Durchlauf für Chunks mit wenigen Relationen
- Zeitleisten-Filter im Explorer (die Daten sind schon da)
- In einer Bank: selbst gehostete oder in der EU gehostete Modelle, Snowflake als Dokumentquelle, Airflow für geplante Re-Ingestion

Danke. Fragen?

Code, Evaluation und Notizen: Repository `graphRAG-demo`

Backup: Ask mit hervorgehobenem Pfad



Backup: Pfadsuche

